ParetoEnsembles.jl: A Julia Package for Multiobjective Parameter Estimation Using Pareto Optimal Ensemble Techniques

Jeffrey D. Varner*
Robert Frederick Smith School of Chemical and Biomolecular Engineering
Cornell University, Ithaca, NY 14853

Abstract

Mathematical models of biological and engineering systems often have many adjustable parameters that must be estimated from multiple, potentially conflicting datasets. Rather than reporting a single best-fit parameter vector, it is often more informative to generate an *ensemble* of parameter sets that collectively map out the trade-offs among competing objectives. ParetoEnsembles.jl is an open-source Julia package that generates such ensembles using Pareto Optimal Ensemble Techniques (POETs), a simulated-annealing-based algorithm that requires no gradient information and is defined entirely by four user-supplied callback functions. This paper describes a substantially rewritten implementation that corrects the original dominance relation, reduces the per-iteration ranking cost through an incremental update scheme, and adds multi-chain parallel execution for improved front coverage and near-linear speedup with thread count. We demonstrate the package on two standard benchmark problems and on a cell-free gene expression model where the ensemble captures the trade-off between fitting messenger RNA and protein time-course data. ParetoEnsembles.jl is available under the MIT license at `https://github.com/varnerlab/ParetoEnsembles.jl`.

Keywords: multiobjective optimization, Pareto front, simulated annealing, ensemble methods, Julia, parameter estimation

1 Introduction

Mathematical models of biological and engineering systems typically contain many adjustable parameters that must be estimated from experimental data. When multiple datasets measure different aspects of the same system — for example, messenger RNA (mRNA) and protein concentrations in a gene expression circuit, or yield and selectivity in a reactor — calibrating parameters to one dataset often degrades the fit to another, giving rise to a multiobjective optimization (MOO) problem [Miettinen, 1999, Deb, 2001, Coello Coello et al., 2007]. Compounding this difficulty, many models exhibit "sloppy" parameter sensitivities: broad, flat valleys in objective space where large regions of parameter space produce nearly equivalent fits [Gutenkunst et al., 2007]. In such settings, reporting a single best-fit parameter vector obscures the practical uncertainty; instead, one needs an *ensemble* of parameter sets that collectively map out the trade-offs among competing objectives. The boundary of this ensemble in objective space is the Pareto front — the set of solutions for which no objective can be improved without degrading at least one other — and the full cloud of near-optimal solutions surrounding it characterizes the range of model behaviors consistent with the available data.

*Corresponding author: `jdv27@cornell.edu`

Several families of algorithms have been developed to approximate Pareto fronts. Population-based evolutionary methods maintain a generation of candidate solutions that are selected, recombined, and mutated toward the front; prominent examples include the Non-dominated Sorting Genetic Algorithm II (NSGA-II) [Deb et al., 2002], its many-objective successor NSGA-III [Deb and Jain, 2014], and the Multiobjective Evolutionary Algorithm Based on Decomposition (MOEA/D), which decomposes the MOO problem into a set of scalar subproblems solved cooperatively [Zhang and Li, 2007]. An alternative trajectory-based strategy is multiobjective simulated annealing (SA), which extends the classical SA framework [Kirkpatrick et al., 1983] to multiple objectives by using Pareto dominance to guide acceptance decisions; early formulations include the Pareto SA method of Czyżak and Jaszkiewicz [1998] and the Archived Multiobjective Simulated Annealing (AMOSA) algorithm of Bandyopadhyay et al. [2008]. Pareto Optimal Ensemble Techniques (POETs), introduced by Song et al. [2010] and later implemented in the Julia programming language as JuPOETs [Bassen et al., 2017], belongs to this SA-based family; POETs ranks the current archive by counting how many other solutions dominate each candidate, then uses the rank as the energy in a Metropolis-type acceptance criterion. Mature software implementations of these and related algorithms are available in several languages, including pymoo in Python [Blank and Deb, 2020] and Metaheuristics.jl in Julia [Mejía-de Dios et al., 2022], both of which focus primarily on population-based evolutionary strategies.

In this study, we present ParetoEnsembles.jl, a substantially rewritten Julia [Bezanson et al., 2017] implementation of the POETs algorithm. The package takes a user-defined model and a set of objective functions — each measuring how well a candidate parameter vector reproduces a particular dataset — and returns a collection of parameter sets spanning the trade-off surface, along with their objective values and dominance ranks. Several algorithmic corrections and performance improvements have been made relative to the original implementation: the dominance check has been corrected from weak to strict Pareto dominance, the per-iteration ranking cost has been reduced from $O(n^2m)$ to $O(nm)$ through an incremental update scheme (where n is the archive size and m is the number of objectives), and multi-chain parallel execution has been added to improve front coverage. We describe the algorithm and its implementation (Sections 2 and 3), demonstrate the package on two standard benchmark problems and a cell-free gene expression model (Section 4), and discuss limitations and future directions (Section 5).

2 Algorithm

Consider an archive of n solutions evaluated on m objectives, where lower values are preferred. Solution $\mathbf{x}_j$ *strictly dominates* solution $\mathbf{x}_i$, written $\mathbf{x}_j \prec \mathbf{x}_i$, if and only if

$$f_k(\mathbf{x}_j) \leq f_k(\mathbf{x}_i) \quad \forall\, k \in \{1, \ldots, m\} \qquad \text{and} \qquad \exists\, k : f_k(\mathbf{x}_j) < f_k(\mathbf{x}_i). \tag{1}$$

The *Pareto rank* of solution $\mathbf{x}_i$ is the number of solutions in the archive that strictly dominate it:

$$R(\mathbf{x}_i) = \big|\{\, j \neq i \mid \mathbf{x}_j \prec \mathbf{x}_i \,\}\big|. \tag{2}$$

Solutions with $R = 0$ lie on the Pareto front. Identical solutions ($\mathbf{f}(\mathbf{x}_j) = \mathbf{f}(\mathbf{x}_i)$ in all objectives) do not dominate each other and therefore share the same rank. It is worth noting that the original implementation [Song et al., 2010, Bassen et al., 2017] used *weak* dominance, checking only that $f_k(\mathbf{x}_j) \leq f_k(\mathbf{x}_i)$ for all k without requiring strict inequality in at least one objective. Under weak dominance, identical solutions count as dominating each other, which inflates their ranks and causes excessive pruning of good solutions from the archive. The current implementation enforces strict Pareto dominance as defined in Equation (1). The pairwise test iterates over objectives

and terminates early when any objective of j is found to be worse than i, avoiding unnecessary comparisons (Algorithm 1).

Algorithm 1 StrictlyDominates($\mathbf{f}_j, \mathbf{f}_i, m$) — Does solution j strictly dominate solution i?

Require: Objective vectors $\mathbf{f}_j, \mathbf{f}_i \in \mathbb{R}^m$ (lower is better)
Ensure: TRUE if $\mathbf{f}_j \prec \mathbf{f}_i$, FALSE otherwise

```
dominated ← TRUE
strictly_better ← FALSE
for k = 1, ..., m do
    if f_{j,k} > f_{i,k} then
        return FALSE                    ▷ j is worse in objective k; cannot dominate
    else if f_{j,k} < f_{i,k} then
        strictly_better ← TRUE          ▷ j is better in at least one objective
    end if
end for
return dominated ∧ strictly_better
```

A central computational concern is the cost of ranking. When a single candidate solution $\mathbf{x}_{\text{new}}$ is appended to an archive of $n-1$ existing solutions with known ranks $\mathbf{R}$, the naïve approach recomputes all $n(n-1)$ pairwise dominance checks at $O(n^2m)$ cost. Because only one solution changed, however, we can instead perform a single $O(nm)$ pass over the existing archive. For each existing solution $\mathbf{x}_i$, we test dominance in both directions: if the new solution dominates $\mathbf{x}_i$ we increment R_i, and if $\mathbf{x}_i$ dominates the new solution we accumulate toward the new solution's rank. This incremental procedure reduces the per-iteration cost from $O(n^2m)$ to $O(nm)$, with a full $O(n'^2m)$ re-rank performed only after archive pruning when n' is small (Algorithm 2).

Algorithm 2 RankInsert($\mathcal{A}, \mathbf{R}$) — Incremental rank update after appending one solution

Require: Archive $\mathcal{A} = \{(\mathbf{x}_1, \mathbf{f}_1), \ldots, (\mathbf{x}_n, \mathbf{f}_n)\}$ where $(\mathbf{x}_n, \mathbf{f}_n)$ is newly appended
Require: Rank array $\mathbf{R} = [R_1, \ldots, R_{n-1}]$ valid for solutions $1, \ldots, n-1$
Ensure: Updated $\mathbf{R} = [R_1, \ldots, R_n]$ valid for the full archive

```
R_new ← 0
for i = 1, ..., n − 1 do
    if StrictlyDominates(f_n, f_i, m) then
        R_i ← R_i + 1                   ▷ new solution dominates i
    end if
    if StrictlyDominates(f_i, f_n, m) then
        R_new ← R_new + 1               ▷ i dominates new solution
    end if
end for
Append R_new to R
```

With the dominance check and incremental ranking in hand, the complete Pareto simulated annealing procedure combines these components into a single outer-inner loop structure (Algorithm 3). The algorithm is seeded with an initial parameter vector $\mathbf{x}_0$, an initial temperature $T = 1$, and an archive containing the single evaluated solution $(\mathbf{x}_0, \mathbf{f}(\mathbf{x}_0))$. At each temperature level, N_{iter} candidate solutions are generated by perturbing the current best parameter vector through a user-supplied neighbor function and evaluating the resulting objectives. Each candidate is temporarily appended to the archive, and the rank array is updated incrementally via RankInsert at $O(nm)$

cost per candidate (line 13). The acceptance probability, computed from the rank array and the current temperature, determines whether the candidate is retained. Upon acceptance, solutions whose rank meets or exceeds the cutoff R_{cutoff} are pruned from the archive, and a full re-rank is performed on the smaller remaining set. If the archive still exceeds a hard size cap $n_{\max}$ after pruning, only the $n_{\max}$ lowest-ranked solutions are kept. Upon rejection, the candidate is immediately removed and the rank array is restored from a saved snapshot (lines 22–23), a strategy we refer to as *pop-on-reject*. In the original implementation, rejected candidates remained in the archive, causing unbounded growth of n between accepted moves and progressively more expensive ranking operations; the pop-on-reject strategy eliminates this problem entirely. After exhausting all iterations at a given temperature, the cooling function reduces the temperature, and the process repeats until T falls below $T_{\min}$.

Two opportunities for parallelism arise from the structure of the algorithm. First, the outer loop of both the full ranking and the incremental RankInsert procedure iterates independently over archive solutions: each iteration reads the shared archive and writes only its own rank entry R_i. Setting `parallel_evaluation=true` dispatches to threaded variants that distribute these independent iterations across available threads using Julia's `Threads.@threads` construct, with an atomic accumulator used for R_{new} in the incremental case since multiple threads contribute to it. Second, and more significantly, because simulated annealing is inherently sequential within a single chain, the most effective parallelization strategy is to run C independent chains from different starting points and merge the resulting archives. Each chain receives its own deterministic random number generator seeded from a base seed and the chain index, ensuring reproducibility (Algorithm 4). After all chains complete, the archives are concatenated and a final full ranking is performed on the merged set. This multi-chain approach provides near-linear speedup with thread count and yields improved Pareto front coverage from the diversity of initial conditions.

3 Software Design

ParetoEnsembles.jl is a pure Julia package with no dependencies beyond the standard library module `Random`. The primary entry point, `estimate_ensemble`, runs a single Pareto SA chain and returns the retained objective values, parameter vectors, and Pareto ranks. Its parallel counterpart, `estimate_ensemble_parallel`, runs C independent chains concurrently, merges their archives, and re-ranks the combined set. A standalone `rank_function` computes Pareto ranks for an arbitrary $m \times n$ objective matrix and can be used independently of the optimization loop (Table 1).

Table 1: Public API of ParetoEnsembles.jl.

Function	**Description**
`estimate_ensemble`	Run a single Pareto SA chain. Returns objective values, parameter vectors, and Pareto ranks for the retained archive.
`estimate_ensemble_parallel`	Run C independent chains in parallel (one per starting point), merge archives, and re-rank.
`rank_function`	Compute Pareto ranks for an $m \times n$ objective matrix.

The algorithm is entirely defined by four user-supplied callback functions. The objective function maps a parameter vector $\mathbf{x} \in \mathbb{R}^d$ to an $m \times 1$ array of objective values (Table 2). The neighbor function perturbs a given parameter vector to generate a candidate; a common choice is

multiplicative Gaussian noise with bound enforcement. The acceptance probability function takes the current rank array and temperature and returns a scalar probability, typically of the Boltzmann form $\exp(-R_{\text{last}}/T)$. Finally, the cooling function maps the current temperature to a reduced value, with geometric schedules of the form $T' = \alpha T$ for $\alpha \in [0.8, 0.95]$ providing a practical balance between exploration and convergence.

Table 2: User-supplied callback functions.

Callback	**Signature**	**Purpose**
`objective_function`	$(\mathbf{x}) \to \mathbb{R}^{m\times 1}$	Evaluate m objectives at parameter vector $\mathbf{x}$
`neighbor_function`	$(\mathbf{x}) \to \mathbb{R}^{d}$	Generate a candidate by perturbing $\mathbf{x}$
`acceptance_probability_function`	$(\mathbf{R}, T) \to [0, 1]$	Acceptance probability given ranks and temperature
`cooling_function`	$(T) \to T'$	Annealing schedule

Beyond the four callbacks, the behavior of `estimate_ensemble` is controlled by several keyword arguments. The `rank_cutoff` parameter (default 5) sets the pruning threshold above which solutions are discarded from the archive, while `maximum_number_of_iterations` (default 20) controls how many candidates are generated at each temperature level. Annealing terminates when the temperature falls below `temperature_min` (default 10^{-4}). The `maximum_archive_size` parameter (default 1000) provides a hard upper bound on archive size as a safety net against pathological growth. Setting `parallel_evaluation` to `true` activates threaded ranking, and an optional `rng` argument accepts any `AbstractRNG` for reproducibility.

4 Examples

We demonstrate ParetoEnsembles.jl on two standard multiobjective benchmarks. The first is the Binh–Korn problem [Binh and Korn, 1997], a constrained two-objective test case defined by

$$\begin{aligned} \min_{\mathbf{x}} \quad & f_1 = 4x_1^2 + 4x_2^2, \qquad f_2 = (x_1 - 5)^2 + (x_2 - 5)^2 \\ \text{s.t.} \quad & (x_1 - 5)^2 + x_2^2 \le 25, \qquad (x_1 - 8)^2 + (x_2 - 3)^2 \ge 7.7 \\ & 0 \le x_1 \le 5, \quad 0 \le x_2 \le 3. \end{aligned} \tag{3}$$

Constraints are enforced via quadratic penalty terms added to the objective values, and bound constraints are handled by clamping in the neighbor function (Listing 1). The same problem can be solved using multi-chain parallel execution, where four chains are launched from diverse starting points to improve coverage of the Pareto front (Listing 2). We ran both benchmarks using ten parallel chains with $N_{\text{iter}} = 60$, $R_{\text{cutoff}} = 12$, and $\alpha = 0.95$. The resulting Pareto fronts in objective space recover the characteristic trade-off curves for both problems, with a visible cloud of near-optimal solutions surrounding the non-dominated front (Figure 1a,b). The computed front for the Fonseca–Fleming case lies directly on the theoretical curve, confirming that the algorithm converges to the correct solution set. In parameter space, the Binh–Korn Pareto-optimal decision vectors trace a curved manifold from the origin toward $(5, 3)$, while the Fonseca–Fleming solutions cluster along the diagonal $x_1 \approx x_2$ as expected from the symmetric structure of the objectives (Figure 1c,d).

Listing 1: Binh–Korn benchmark with ParetoEnsembles.jl.

```julia
using ParetoEnsembles

function objective_function(x)
    f = zeros(2, 1)
    f[1] = 4.0 * x[1]^2 + 4.0 * x[2]^2
    f[2] = (x[1] - 5)^2 + (x[2] - 5)^2
    # penalty for constraint violations
    lambda = 100.0
    v1 = 25 - (x[1] - 5)^2 - x[2]^2
    v2 = (x[1] - 8)^2 + (x[2] - 3)^2 - 7.7
    f[1] += lambda * (min(0, v1))^2
    f[2] += lambda * (min(0, v2))^2
    return f
end

neighbor_function(x) = clamp.(
    x .* (1 .+ 0.05 * randn(length(x))),
    [0.0, 0.0], [5.0, 3.0])

acceptance_probability_function(R, T) =
    exp(-R[end] / T)

cooling_function(T) = 0.9 * T

(EC, PC, RA) = estimate_ensemble(
    objective_function, neighbor_function,
    acceptance_probability_function, cooling_function,
    [2.5, 1.5]; rank_cutoff=4.0,
    maximum_number_of_iterations=40, show_trace=false)

pareto_idx = findall(RA .== 0)
```

The second benchmark is the Fonseca–Fleming problem [Fonseca and Fleming, 1995], an unconstrained test case with d-dimensional decision variables:

$$f_1(\mathbf{x}) = 1 - \exp\left(-\sum_{i=1}^{d}\left(x_i - \tfrac{1}{\sqrt{d}}\right)^2\right), \qquad f_2(\mathbf{x}) = 1 - \exp\left(-\sum_{i=1}^{d}\left(x_i + \tfrac{1}{\sqrt{d}}\right)^2\right), \tag{4}$$

with $x_i \in [-4, 4]$. The theoretical Pareto front is obtained by setting all components $x_i = t/\sqrt{d}$ for $t \in [-1, 1]$, yielding a known parametric curve that provides a closed-form reference for validation. We set $d = 3$ and use the same annealing parameters as the Binh–Korn example. The neighbor function applies additive Gaussian perturbation with clamping to the box constraints.

We compared a single chain started at $(2.5, 1.5)$ against ten parallel chains on the Binh–Korn problem to illustrate the effect of multi-chain execution (Figure 2). The single chain recovers the trade-off curve but, because every retained solution is non-dominated within that chain's archive, the rank array contains only rank-zero entries. After merging the ten-chain archives and re-ranking the combined set, the multi-chain run produces a denser front with broader coverage of the feasible region, demonstrating how diverse starting points improve exploration of the trade-off surface.

Listing 2: Multi-chain parallel execution on the Binh–Korn problem (requires `julia -t4`).

```julia
using ParetoEnsembles

# ... (same callbacks as Listing 1) ...

initial_states = [
    [2.5, 1.5], [0.5, 2.5],
    [4.0, 0.5], [1.0, 1.0]
]

(EC, PC, RA) = estimate_ensemble_parallel(
    objective_function, neighbor_function,
    acceptance_probability_function, cooling_function,
    initial_states;
    rank_cutoff=4.0, maximum_number_of_iterations=40,
    show_trace=false, rng_seed=42)

println("Solutions: ", size(EC, 2))
println("Pareto-optimal: ", count(RA .== 0))
```

As a more realistic application, we considered the estimation of kinetic parameters for a cell-free gene expression circuit. The model describes the production of deGFP protein driven by the endogenous sigma factor σ_{70} via the P70 promoter, following the effective biophysical framework of Adhikari et al. [2020]. The system is governed by two ordinary differential equations for mRNA (m) and protein (p):

$$\dot{m} = \alpha\, u(\sigma_{70}) - \delta_m\, m, \tag{5}$$

$$\dot{p} = \kappa\, m\, w(t) - \delta_p\, p, \tag{6}$$

where $u(\sigma_{70}) = \sigma_{70}^n/(K^n + \sigma_{70}^n)$ is a Hill-type promoter activity function and $w(t) = \exp(-\ln 2 \cdot t/\tau_{1/2})$ captures the experimentally observed decay of translational capacity over the course of the cell-free reaction. Five parameters were treated as unknown: the maximum transcription rate α, the effective translation rate constant κ, the mRNA degradation rate δ_m, the promoter dissociation constant K, and the translation capacity half-life $\tau_{1/2}$. The remaining parameters ($\sigma_{70} = 35$ nM, $n = 1.5$, $\delta_p = 0.005$ h^{-1}) were fixed at values taken from the literature [Adhikari et al., 2020].

We generated synthetic experimental data by simulating the model with known "true" parameter values and adding multiplicative Gaussian noise (coefficient of variation $\approx 12\%$) at twelve time points spanning 0–16 h, mimicking the sampling protocol of typical cell-free protein synthesis experiments. Two objectives were defined: the normalized sum-of-squared error between simulated and observed mRNA concentrations ($\varepsilon_{\mathrm{mRNA}}$), and the corresponding error for protein concentrations ($\varepsilon_{\mathrm{protein}}$). We ran ten parallel chains with $R_{\mathrm{cutoff}} = 8$ and $N_{\mathrm{iter}} = 50$, starting from random initial parameter vectors drawn uniformly within the bounds.

The resulting ensemble of 191 parameter sets (rank ≤ 1) captured the dynamics of both mRNA and protein expression (Figure 3a,b). The 95% confidence interval of the ensemble simulations bracketed the synthetic data at nearly all time points, and the ensemble mean tracked the underlying trajectory. The Pareto front revealed a clear trade-off between fitting the mRNA and protein time courses (Figure 3c): parameter sets that minimized $\varepsilon_{\mathrm{mRNA}}$ tended to have slightly larger $\varepsilon_{\mathrm{protein}}$, and vice versa. This trade-off arises naturally because the mRNA degradation rate and translation

capacity half-life have opposing effects on the two species, and the noise in each measurement channel pulls the optimal parameters in different directions. The ensemble provides decision makers with a family of plausible parameter sets rather than a single point estimate, enabling downstream uncertainty quantification of model predictions.

To quantify the benefit of multi-chain parallelism, we measured wall clock time for the cell-free ensemble estimation (ten chains, $N_{\text{iter}} = 50$) as a function of the number of Julia threads (Table 3). Each configuration was run five times and the median time is reported. The speedup is near-linear up to four threads (3.1×) and reaches 4.1× at eight threads, with diminishing returns due to load imbalance when ten chains are distributed across eight threads and the serial cost of the final archive merge and re-ranking step.

Table 3: Multi-chain parallel scaling on the cell-free gene expression model (ten chains, $N_{\text{iter}} = 50$, median of five runs).

Threads	**Wall clock (s)**	**Speedup**
1	18.52	1.0×
2	9.38	2.0×
4	5.94	3.1×
8	4.51	4.1×

5 Discussion

ParetoEnsembles.jl provides a lightweight, dependency-free Julia implementation of Pareto Optimal Ensemble Techniques for multiobjective optimization. Relative to the original implementation [Song et al., 2010, Bassen et al., 2017], the current version corrects the dominance relation from weak to strict, reduces the per-iteration ranking cost from $O(n^2m)$ to $O(nm)$ through incremental updating, bounds archive growth through the pop-on-reject strategy and a hard size cap, and adds both threaded ranking and multi-chain parallel execution. The callback-based design decouples the optimization algorithm from the problem definition, allowing users to apply ParetoEnsembles.jl to any domain where objective functions can be evaluated pointwise.

Population-based evolutionary methods such as NSGA-II [Deb et al., 2002] and MOEA/D [Zhang and Li, 2007], available through mature frameworks like pymoo [Blank and Deb, 2020] and Metaheuristics.jl [Mejía-de Dios et al., 2022], are generally more sample-efficient than simulated annealing for smooth, well-conditioned objective landscapes, and should be preferred when the problem structure supports them. ParetoEnsembles.jl targets a complementary use case: problems where the user wants a simple, gradient-free ensemble generator with minimal configuration — four callback functions and a handful of keyword arguments — and where the primary goal is to characterize the cloud of near-optimal parameter sets rather than to find the tightest possible approximation of the Pareto front. The SA-based approach is particularly convenient for black-box objectives arising in systems biology, where function evaluations may be expensive, noisy, or non-differentiable, and where the simplicity of the interface lowers the barrier to generating parameter ensembles as part of a modeling workflow.

Several features are deliberately omitted from the current scope. The package does not implement gradient-based refinement, surrogate modeling, or adaptive cooling schedules, nor does it provide built-in visualization or benchmark problem libraries. Integration with Julia's automatic differentiation ecosystem, for example through ForwardDiff.jl or Enzyme.jl, could enable hybrid

gradient-free/gradient-based strategies for problems where gradients are available for some but not all objectives. Another promising direction is adaptive archive management using hypervolume indicators [Zitzler et al., 2003] in place of simple rank-based pruning, which would provide more principled control over the distribution of retained solutions along the Pareto front.

Acknowledgments

J.D.V. acknowledges support from the National Heart, Lung, and Blood Institute (NHLBI) of the National Institutes of Health (NIH) under awards R33 HL141787 and R01 HL71944.

Data and Code Availability

ParetoEnsembles.jl is open source under the MIT license. The source code is available at `https://github.com/varnerlab/ParetoEnsembles.jl`. Documentation is hosted at `https://varnerlab.github.io/ParetoEnsembles.jl/dev/`. The package can be installed directly from the repository:

```
using Pkg
Pkg.add(url="https://github.com/varnerlab/ParetoEnsembles.jl")
```

Example code reproducing all results in this paper is included in the `paper/code` directory of the repository.

References

Abhinav Adhikari, Michael Vilkhovoy, Sandra Vadhin, Ha Eun Lim, and Jeffrey D. Varner. Effective biophysical modeling of cell free transcription and translation processes. *Frontiers in Bioengineering and Biotechnology*, 8:539081, 2020. doi: 10.3389/fbioe.2020.539081.

Sanghamitra Bandyopadhyay, Sriparna Saha, Ujjwal Maulik, and Kalyanmoy Deb. A simulated annealing-based multiobjective optimization algorithm: AMOSA. *IEEE Transactions on Evolutionary Computation*, 12(3):269–283, 2008. doi: 10.1109/TEVC.2007.900837.

David Bassen, Michael Vilkhovoy, Mason Minot, Jonathan Butcher, and Jeffrey D. Varner. JuPOETs: A constrained multiobjective optimization approach to estimate biochemical model ensembles in the Julia programming language. *BMC Systems Biology*, 11(1):10, 2017. doi: 10.1186/s12918-016-0380-2.

Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017. doi: 10.1137/141000671.

To Thanh Binh and Ulrich Korn. MOBES: A multiobjective evolution strategy for constrained optimization problems. In *Proceedings of the Third International Conference on Genetic Algorithms (Mendel 97)*, pages 176–182, 1997.

Julian Blank and Kalyanmoy Deb. pymoo: Multi-objective optimization in Python. *IEEE Access*, 8:89497–89509, 2020. doi: 10.1109/ACCESS.2020.2990567.

Carlos A. Coello Coello, Gary B. Lamont, and David A. Van Veldhuizen. *Evolutionary Algorithms for Solving Multi-Objective Problems*. Springer, second edition, 2007. doi: 10.1007/978-0-387-36797-2.

Piotr Czyżak and Andrzej Jaszkiewicz. Pareto simulated annealing—a metaheuristic technique for multiple-objective combinatorial optimization. *Journal of Multi-Criteria Decision Analysis*, 7 (1):34–47, 1998. doi: 10.1002/(SICI)1099-1360(199801)7:1⟨34::AID-MCDA161⟩3.0.CO;2-6.

Kalyanmoy Deb. *Multi-Objective Optimization Using Evolutionary Algorithms*. John Wiley & Sons, 2001.

Kalyanmoy Deb and Himanshu Jain. An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: Solving problems with box constraints. *IEEE Transactions on Evolutionary Computation*, 18(4):577–601, 2014. doi: 10.1109/TEVC.2013.2281535.

Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2): 182–197, 2002. doi: 10.1109/4235.996017.

Carlos M. Fonseca and Peter J. Fleming. An overview of evolutionary algorithms in multiobjective optimization. *Evolutionary Computation*, 3(1):1–16, 1995. doi: 10.1162/evco.1995.3.1.1.

Ryan N. Gutenkunst, Joshua J. Waterfall, Fergal P. Casey, Kevin S. Brown, Christopher R. Myers, and James P. Sethna. Universally sloppy parameter sensitivities in systems biology models. *PLoS Computational Biology*, 3(10):e189, 2007. doi: 10.1371/journal.pcbi.0030189.

Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671.

Jesús-Adolfo Mejía-de Dios, Efrén Mezura-Montes, and Carlos A. Coello Coello. Metaheuristics.jl: Towards reproducible optimization research. *Journal of Open Source Software*, 7(78):4723, 2022. doi: 10.21105/joss.04723.

Kaisa Miettinen. *Nonlinear Multiobjective Optimization*. Springer, 1999. doi: 10.1007/978-1-4615-5563-6.

Satyaprakash Song, Anirikh Chakrabarti, Jay Bhatt, Devendranath Bhatt, and Jeffrey D. Varner. Identifying ensembles of signal transduction models using Pareto optimal ensemble techniques (POETs). *Biotechnology Journal*, 5(7):768–780, 2010. doi: 10.1002/biot.201000059.

Qingfu Zhang and Hui Li. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6):712–731, 2007. doi: 10.1109/TEVC.2007.892759.

Eckart Zitzler, Lothar Thiele, Marco Laumanns, Carlos M. Fonseca, and Viviane Grunert da Fonseca. Performance assessment of multiobjective optimizers: An analysis and review. *IEEE Transactions on Evolutionary Computation*, 7(2):117–132, 2003. doi: 10.1109/TEVC.2003.810758.

Algorithm 3 ParetoEnsembles — Pareto simulated annealing with incremental ranking

Require: User callbacks: Objective, Neighbor, Accept, Cool
Require: Initial parameter vector $\mathbf{x}_0$; hyperparameters N_{iter}, R_{cutoff}, $T_{\min}$, $n_{\max}$
Ensure: Archive of solutions with objective values, parameters, and Pareto ranks

```
x_best ← x_0
A ← {(x_0, Objective(x_0))}                          ▷ seed archive
R ← [0]                                              ▷ initial solution has rank 0
T ← 1

while T > T_min do                                   ▷ outer loop: anneal
    for t = 1, ..., N_iter do                        ▷ inner loop: generate candidates

        x' ← Neighbor(x_best)
        f' ← Objective(x')

        R_saved ← copy(R)                            ▷ snapshot for rollback
        Append (x', f') to A
        RankInsert(A, R)                             ▷ O(nm) incremental update (Algorithm 2)

        p ← Accept(R, T)
        if p > U(0,1) then                           ▷ accept: prune and update
            A ← {(x_i, f_i) ∈ A | R_i < R_cutoff}    ▷ remove high-rank solutions
            R ← FullRank(A)                          ▷ re-rank pruned archive
            if |A| > n_max then                      ▷ archive cap: keep best
                Keep only the n_max solutions with lowest R_i
                R ← FullRank(A)
            end if
            x_best ← x'
        else                                         ▷ reject: pop and rollback
            Remove (x', f') from A
            R ← R_saved
        end if
    end for

    T ← Cool(T)
end while

R ← FullRank(A)                                      ▷ final ranking
return A, R
```

Algorithm 4 ParetoEnsemblesParallel — Multi-chain parallel execution

Require: Initial states $\{\mathbf{x}_0^{(1)}, \ldots, \mathbf{x}_0^{(C)}\}$; base seed s
Ensure: Merged archive with Pareto ranks

for $c = 1, \ldots, C$ **in parallel do** ▷ one thread per chain
  $\mathrm{rng}_c \leftarrow \textsc{Seed}(s, c)$ ▷ deterministic per-chain RNG
  $(\mathcal{A}_c, \mathbf{R}_c) \leftarrow \textsc{ParetoEnsembles}(\mathbf{x}_0^{(c)}, \mathrm{rng}_c)$
end for

$\mathcal{A} \leftarrow \mathcal{A}_1 \cup \cdots \cup \mathcal{A}_C$ ▷ merge archives
$\mathbf{R} \leftarrow \textsc{FullRank}(\mathcal{A})$ ▷ re-rank merged archive
return $\mathcal{A}$, $\mathbf{R}$

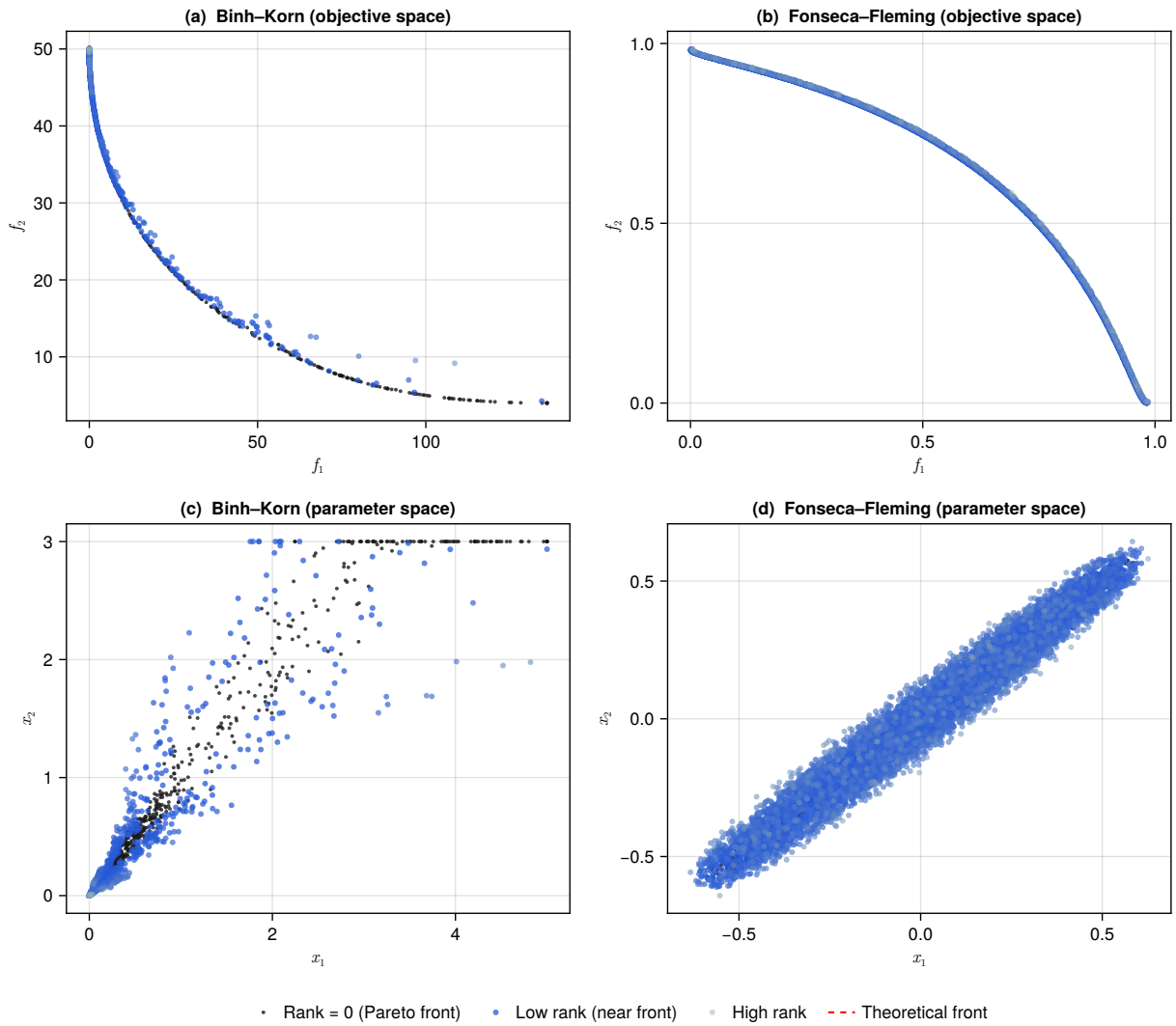

Figure 1: Benchmark results for (a,c) Binh–Korn and (b,d) Fonseca–Fleming ($d = 3$). Top row: objective space; bottom row: parameter-space projections (x_1 vs. x_2). Non-dominated solutions (rank $= 0$, dark) define the Pareto front, while near-optimal solutions (blue, colored by rank) form a cloud around it representing the retained ensemble. The dashed red curve in (b) is the theoretical Pareto front. Ten parallel chains were used with $R_{\text{cutoff}} = 12$, $N_{\text{iter}} = 60$, and $\alpha = 0.95$.

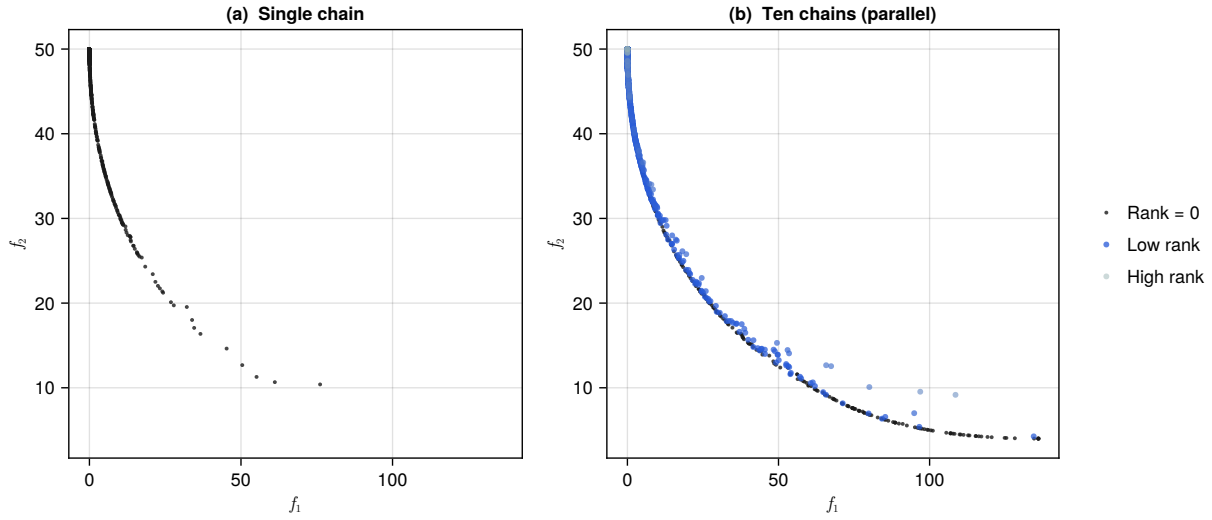

Figure 2: Comparison of (a) single-chain and (b) ten-chain parallel execution on the Binh–Korn problem. Both panels share the same axis limits. The single chain yields a sparse archive of non-dominated solutions, while the merged multi-chain archive provides denser front coverage and a visible cloud of near-optimal solutions (blue) surrounding the front.

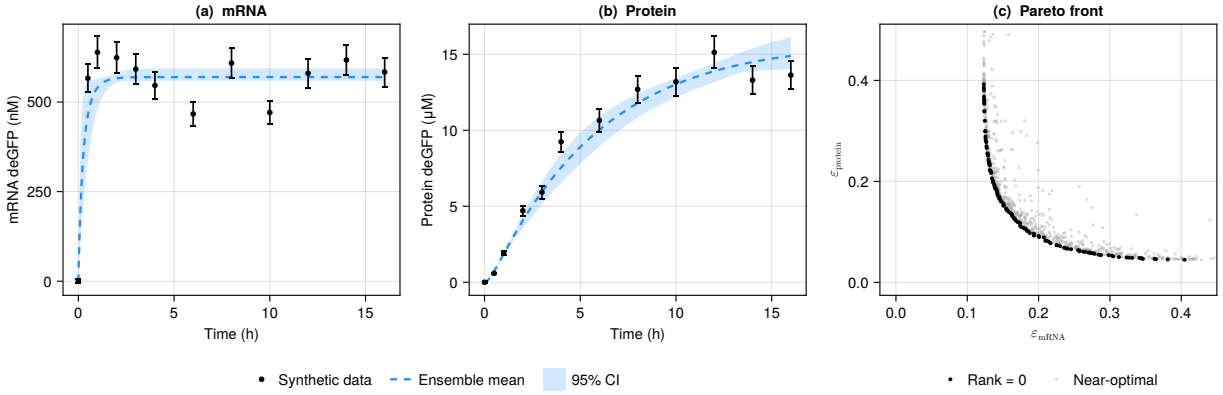

Figure 3: Ensemble estimation for a cell-free gene expression circuit ($\sigma_{70} \rightarrow$ P70 $\rightarrow$ deGFP). (a) mRNA and (b) protein concentration versus time; the blue dashed line is the ensemble mean, the shaded region is the 95% confidence interval, and black points are synthetic data with error bars. (c) Pareto front showing the trade-off between mRNA and protein fitting error.